

Internship III Report

On

Development and Testing of the Life Programmable Interface (LPI) Platform

Submitted to

Amity University Uttar Pradesh



in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Artificial Intelligence

By

**Yashika Verma
(A023119823014)**

under the guidance of

Dr. Mansi Sharma

**AMITY SCHOOL OF ENGINEERING AND
TECHNOLOGY DOMAIN OF ENGINEERING AND
TECHNOLOGY AMITY UNIVERSITY UTTAR PRADESH**

July 2026

DECLARATION

I, **Yashika Verma**, a student of B.Tech **Artificial Intelligence** hereby declare that the Internship III project titled “**Development and Testing of the Life Programmable Interface (LPI) Platform**” which is submitted by me to Amity School of Engineering and Technology, Domain of Engineering and Technology, Amity University Uttar Pradesh, in partial fulfillment of requirement for the award of the degree of Bachelor of Technology Artificial Intelligence, has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition.

Noida

Date

Name and Signature of Student

CERTIFICATE

Based on declaration submitted by **Yashika Verma**, student of B. Tech Artificial Intelligence, I hereby certify that the Internship III project titled “**Development and Testing of the Life Programmable Interface (LPI) Platform**” which is submitted to Amity School of Engineering and Technology, Domain of Engineering and Technology, Amity University Uttar Pradesh, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology Artificial Intelligence, is an original contribution with existing knowledge and faithful record of work carried out by him/them under my guidance and supervision.

To the best of my knowledge this work has not been submitted in part or full for any Degree or Diploma to this University or elsewhere.

Noida

Date

(Dr. Mansi Sharma)
(Asst. Professor II)
Amity School of Engineering and Technology
Domain of Engineering and Technology
Amity University Uttar Pradesh

TABLE OF CONTENTS

S. No	Topic	Page No.
I.	Declaration	2
II.	Certificate	3
III.	Table Of Contents	4
IV.	Abstract	5
1.	Introduction	6
2.	Literature Review	8
3.	Methodology	11
4.	Result and Discussions	15
5.	Conclusion and Future Work	18
6.	References	20

Abstract

This report outlines the tasks performed during my 61 days' internship at WINNIIO AB as an Applied AI Intern. The internship was conducted in the fields of Artificial Intelligence and Software Engineering and was mostly aimed at developing, testing, documenting, and integrating the Life Programmable Interface (LPI) Platform – a backend in the LifeAtlas ecosystem intended for users goal management, processing of activity signals, and generating AI-based recommendations based on the SMILE lifecycle model. As a part of the internship, I got familiarized with the platform architecture, set up the local development environment, tested REST APIs, performed backend testing, took part in the quality assurance process, prepared technical documentation, and helped to integrate platform functionalities into different development modules. The LPI platform was created using such technologies as FastAPI, Supabase (PostgreSQL), Docker, Git, GitHub, pytest, and LangGraph. The contributions to the enhancement of automated test coverage, testing and validation of Activity Signals and Recommendation Engine workflows, improvement of recommendation feedback persistence with Supabase, ZeroClaw and GitHub webhooks integration, and assistance in setting up repositories and writing documentation were made during the internship period. They were demonstrated by means of validated pull requests for improvements to SMILE testing, expanded test coverage of Activity Signals with support of time range query, and Recommendation Feedback Persistence with Supabase Storage. The internship was finished with participating in the presentation of the final project that involved showing the finished product and its main features. The internship provided good practical experience in developing backend applications on a large scale, in API-centric architecture, working in a team on software engineering, testing and documentation.

1. Introduction

1.1 Background of the Domain

The fast development of Artificial Intelligence and cloud computing has revolutionized the approach to the design, development, and implementation of the modern software application. In today's world, companies tend to use intelligent back-end systems that are capable of processing large amounts of structured information, making personalized suggestions, automating the decision-making process, and integrating with other external systems. As back-end systems become more complex, there is an increased need for scalable architecture, secure data management, and software engineering.

Contemporary AI-based applications not only consist of machine learning algorithms but also use backend frameworks, databases, cloud technologies, automatic testing, and collaborative development processes. Stability of these applications is ensured by appropriate architecture, sufficient testing, data persistence, user authentication, and continuous integration techniques. That is why backend software engineering has become an integral part of developing intelligent applications.

Life Programmable Interface (LPI) Platform by WINNIIO AB, which has been built as a part of the LifeAtlas ecosystem, is a great example of a modern backend platform that leverages AI capabilities. This platform offers a modular backend architecture that allows building a user-friendly environment where people can create and manage their goals, input structured activity signals, and get personalized recommendations via the SMILE lifecycle model. The platform consists of three core modules: Goal Registry, Activity Signals, and Recommendation Engine that interact with each other providing a cohesive process of intelligent goal management and recommendation generation.

The increasing use of AI-based backend platforms has made the collaboration process in software development more important than ever. For modern software development, versioning, automatic testing, continuous integration, technical documentation, debugging, and modularization of the software are needed in order to ensure high-quality and maintainable software. These concepts have become crucial for prospective software engineers that will work with Artificial Intelligence and backend applications.

Interest in the field of AI, back-end software engineering, and intelligent recommendation systems was the driving force behind my decision to apply for this internship. Interning in this field gave me the chance to engage with actual production-oriented software projects as well as get experience in various aspects of back-end development like API validation and software testing.

1.2 Objectives of the Internship

- To get hands-on experience in building and maintaining backend applications powered by artificial intelligence.
- To learn about the architecture and workflow of the Life Programmable Interface (LPI) Platform.
- To set up and use a local development environment with FastAPI, Docker, and Supabase.
- To learn about the implementation of the Goal Registry, Activity Signals, and Recommendation Engine modules.
- To learn how to build APIs in FastAPI and how to validate and test them with pytest.

- To contribute to the software quality assurance process through debugging, integration tests, and automated test writing.
- To collaborate on software development through Git, GitHub, pull requests, and code review.
- To write technical documentation and developer setup instructions for the platform.
- To support the integration and validation of the platform's modules, including ZeroClaw integration and working with repositories.
- To improve my problem-solving, debugging, and software engineering skills through participation in an actual AI software development project.

1.3 Scope of the Internship

The scope of this internship was mainly focused on the backend software development and quality assurance tasks performed during the development of the Life Programmable Interface (LPI) Platform, a backend software in the LifeAtlas system ecosystem created at WINNIIO AB. The platform aims to provide a means of intelligent goal management, activity structuring and generating recommendations with the help of an AI, through a modular API based architecture.

The tasks included gaining knowledge about the platform architecture and workflow, setting up and configuration of the local development environment, verification of backend APIs, automated testing, debugging of application problems, technical documentation preparation, and support for integration of the platform modules. The practical knowledge was obtained regarding the usage of such technologies as FastAPI, Supabase, Docker, Git, GitHub, pytest, and modern software engineering practices like feature branching, pull requests, etc.

My tasks as an Applied AI intern included performing testing tasks, conducting quality assurance, writing documentation, backend validation, and carrying out feature-specific development tasks assigned by the project team. In the course of the internship, I was engaged in extending automated test coverage of the Activity Signals module, enhancing SMILE test cases, developing Recommendation Feedback Persistence with Supabase, helping in validation of ZeroClaw integration, conducting GitHub webhook tests, and writing developer documentation in order to make the repository easier to onboard and maintain.

The development of an entire platform independently is not included in the internship activities. However, the internship provides the experience in working on an existing production-focused software project where different developers were in charge of different modules of the system. This has allowed me to gain an understanding of backend architectures on a large scale, API development, software testing, debugging, and software engineering workflows in industry.

The skills and knowledge that I acquired through this internship gave me a solid base in backend application development, collaborative software engineering, artificial intelligence-enabled systems, and quality assurance methodologies, and at the same time improved my contribution in real-life software development projects.

2. Literature Review

2.1 Overview of Backend Application Development

Backend application development is concerned with the creation of server-based software designed for processing the business logic, managing data storage, processing the requests from clients, and coordinating communication between various software modules. As opposed to frontend applications, which emphasize user interaction, the backend applications deliver essential functionalities for the current software platforms through providing APIs, authentication, request processing, and database access.

The fast development of Artificial Intelligence and cloud-native applications made the need for scalable backend architectures very important. Contemporary AI-powered platforms usually have modular backend solutions that unite data management, intelligent decision-making, authentication, and API integrations into one platform. Modular backend architectures make integration easier, increase scalability, and maintainability of the systems while making different services independent from each other.

Modular backend architecture of Life Programmable Interface (LPI) Platform that I created during my internship consists of three main modules: Goal Registry, Activity Signals, and Recommendation Engine. These modules use REST APIs for communication and employ Supabase (PostgreSQL) as the main data persistence engine.

2.2 RESTful API Architecture

Representation State Transfer (REST) is among the most popular architectures used when building distributed systems. Through the use of RESTful APIs, clients can communicate with servers using HTTP methods like GET, POST, PATCH, and DELETE, making it possible for systems built on various technologies to exchange information seamlessly.

The REST architecture creates loose coupling in the system by providing standardized endpoints that abstract the functions of the backend. This makes it scalable and allows the integration of third-party services into the system.

The functionalities of the LPI Platform are provided by REST APIs created using FastAPI. Each function of the platform, including managing goals, ingestion of activity signals, recommendation, collection of recommendations, and administration tasks, is handled through dedicated API endpoints.

2.3 FastAPI Framework

FastAPI is a contemporary framework built for Python that allows users to create high-performing RESTful web applications. Its key distinguishing features include support for type hinting in Python and automatic generation of interactive documentation following the OpenAPI standard using tools like Swagger UI and ReDoc.

As compared to conventional backend frameworks, FastAPI offers asynchronous processing, data validation using Pydantic, dependency injection, and an optimized routing system. All these contribute towards decreasing developers' efforts and improving the performance and maintainability of applications.

As part of the internship, FastAPI was used as the basis for implementing the backend of the LPI

Platform. Specifically, it was employed to create the APIs for the modules like Goals, Activity Signals, Recommendations, GitHub Webhooks, and Team Metrics.

2.4 Database Management using Supabase

Supabase is an open source Backend-as-a-Service (BaaS) platform based on PostgreSQL. Supabase is a platform that provides database services, authentication services, storage facilities, row level security features, and RESTful APIs to enable easy development of backend applications.

One of the most important strengths of Supabase is the ability to integrate seamlessly with PostgreSQL with developer-friendly services for authentication, migrations, and APIs. Supabase is thus appropriate for applications that require storage of structured data along with scalable backend services.

The LPI platform uses Supabase as its main persistence layer for storage of goals, activity signals, recommendation feedback, activity logs, system logs, and other application data. Database migrations, authentication, and data persistence in the platform are done through the use of Supabase services.

2.5 Containerization using Docker

Docker is a container technology platform that allows applications and their dependencies to be encapsulated in containers. Containerization provides a similar development environment regardless of the operating system, while making the deployment process easier.

By running applications in containers, developers eliminate possible problems due to varying operating systems configurations and software version mismatches. Docker also facilitates collaboration through provision of the same execution environment for each developer.

While working on the internship, Docker was utilized to set up and run the local development environment, especially those services of Supabase needed by the backend application.

2.6 Software Testing Methodologies

Software testing is an organized and well-planned verification process of making sure that the software adheres to the required functional and non-functional aspects. Efficient testing improves the quality of the software through identification of implementation faults, integration problems, and unanticipated system behaviors before actual deployment. Some of the commonly applied testing processes in modern backend applications include:

Unit Testing - Unit testing tests the accuracy of each unit in the software, which includes testing of functions and classes individually. This form of testing helps developers to test individual code units before integration.

Integration Testing - Integration testing tests how the interaction among several software units is performed, including testing of APIs, databases, authentication, and other external services. The testing makes sure that the independently designed modules work appropriately together.

Regression Testing - Regression testing makes sure that the changes made during development do not affect previous functionalities. This is very useful for collaborative software where different developers are working together.

LPI Platform mainly used automated testing of the software with pytest in which tests were conducted on backend APIs, database, authentication, recommendation, and activity signals processing. Integration testing was vital for the validation of the interaction among different platform modules.

2.7 Version Control and Collaborative Software Development

The purpose of using version control systems is to be able to control the changes made to source code, track development history, and work collaboratively. Git has emerged as the most popular tool used for distributed version control whereas GitHub is one of the platforms that help in collaborative development of software applications.

Usually, the processes of feature branching, peer review, automated testing, and controlled merging to common branches make up collaborative workflows that help in improving the quality of software and avoiding merge conflicts.

In the course of my internship experience, collaborative workflow using GitHub was implemented when developing the LPI Platform as well as feature branches, pull requests, staging, and peer reviews were involved.

2.8 AI-Assisted Recommendation Systems

Recommendation engines are advanced computational modules that make use of the available information in order to offer relevant recommendations to users. Classic recommendation engines used either collaborative or content-based recommendation methods, while current AI-enhanced recommendation engines employ the use of structured information about the user, contextual reasoning, and machine learning methods.

As far as the LPI Platform is concerned, recommendation creation is carried out with the help of a combination of information from the Goal Registry module and the Activity Signals module using the SMILE lifecycle pattern. The use of deterministic reasoning, LangGraph orchestration and several fallback methods allows reliable recommendation creation in the LPI Platform.

2.9 Software Engineering Practices

The modern software engineering practice focuses on maintainable code, modularity, testing, documentation, and collaboration. The use of code review, technical documentation, testing, and project structure helps achieve better maintenance of software and minimizes any defect in the software.

In the entire duration of the internship, there was consistent application of software engineering practices in the backend programming, testing, debugging, documentation, and reviewing of the pull requests. It was a good experience to work in a software engineering process in actual AI software development projects.

3. Methodology

3.1 System Architecture

The LPI Platform has been designed as a modular backend application that allows users to create and manage goals, capture activity signals, and receive AI-generated recommendations via a single REST API interface. The platform is built based on the concept of API-first architecture, where each module operates independently yet uses the same persistent layer (Supabase/PostgreSQL). The backend framework used in the development is FastAPI which offers high-performance API endpoints, automated request validation, and Swagger API documentation. The overall architecture of the platform includes three main modules:

Goal Registry - This module is responsible for creation, update, retrieval, and management of user goals as well as tracking of the progress of their execution based on the SMILE lifecycle.

Activity Signals - This module is used for capturing structured activity events from various sources and building a timeline of user activities.

Recommendation Engine - This module performs analysis of goals and activity signals in order to deliver AI recommendations with the help of deterministic algorithms and LangGraph pipeline.

All the modules interact through REST API interfaces and a common database layer.

3.2 Development Environment

A consistent local development environment was adopted for the entire duration of the internship. The project dependencies were handled in Python, whereas the backend application was launched using FastAPI. Database services were offered through a local installation of Supabase running in Docker containers. The development environment was comprised of the following components:

Component	Purpose
Python 3.11	Backend application development
FastAPI	REST API framework
Supabase	PostgreSQL database and authentication
Docker	Local containerized services
Git & GitHub	Version control and collaboration
pytest	Automated testing
Swagger UI	API validation and documentation

Setting up the development environment involved cloning the repository, setting up the project's dependencies, configuring environment variables, running the local Supabase instance, applying the database migrations and running the FastAPI app. After successful execution of the server, the auto-generated Swagger documentation was used to check the availability of the API.

3.3 Development Workflow

Development of the LPI Platform was carried out through a collaborative software development

process which allowed simultaneous contributions by several developers without compromising the integrity of the code and project stability. The workflow used in the development of the project involved the following steps:

1. Cloning of the current state of the repository and synchronization with the staging branch.
2. Setup of the local environment and configuration of the databases.
3. Creation of a specific branch for the assigned work.
4. Changes of implementation of features and fixing bugs as required.
5. Running of tests to ensure proper functionality.
6. Manual testing of APIs with Swagger UI.
7. Technical documentation creation when needed.
8. Submission of the changes via a Pull Request on GitHub.
9. Peer review and incorporation of feedback.
10. Merging the reviewed changes with the staging branch.

This process facilitated collaborative development and increased code maintainability.

3.4 Repository Structure

The project repository was organized in a modular manner, which helped in keeping the business logic, routing, tests, database configuration, documentation, and frontend resources separated from each other. It helped to simplify the process of maintaining the code and make it possible to develop various modules separately. Major repository components are briefly listed below.

Directory	Purpose
<code>src/lpi</code>	Core backend application
<code>routers</code>	REST API endpoints
<code>middleware</code>	Authentication and request processing
<code>store.py</code>	Database interaction layer
<code>tests</code>	Automated test suites
<code>supabase</code>	Database migrations and configuration

docs

Technical documentation

frontend-js

React-based frontend application

Organizing various functions in separate modules made it easier to read and maintain the code without duplication. There were also various configuration files like `pyproject.toml`, `Makefile`, `.env.example`, and pre-commit hooks within the repository.

3.5 Testing Methodology

Testing was an important part of the software development life cycle during the internship period. Rather than performing only manual tests, the platform used an automated testing strategy using `pytest`, supplemented with manual testing using Swagger UI. The following steps were involved in the testing process.

1. API Testing - The backend endpoints were tested to confirm the appropriate handling of requests and responses, their status codes, format and the authentication process. API testing was conducted for Goal Registry, Activity Signals, Recommendation Engine, Recommendation Feedback and other backend services.
2. Integration Testing - This testing ensured that all components of the platform worked effectively in conjunction with one another. This kind of testing included validating interactions between FastAPI endpoints, Supabase database queries, authentication middlewares and recommendation workflows.
3. Regression Testing - After adding new features or resolving bugs, regression testing was done to check that the existing features were not broken. Test suits were run before sending pull requests to check any unwanted effects introduced.
4. Manual Validation - Automated tests were complemented with manual validation of the endpoints using Swagger UI.

The testing strategy contributed to improving software reliability, identifying defects during development, and maintaining platform stability throughout the internship.

3.6 Documentation Workflow

Documentation was carried out in parallel to development in order to enhance maintainability and onboarding of developers. Documentation was done during the entire period of the internship in case any feature, test, or development was introduced. Documentation consisted of:

- Repository configuration guide.
- Setting up the local development environment.
- API documentation through Swagger UI.
- Test documentation.
- Documentation for pull requests.
- QA Documentation.
- Documentation for integration of the platform components.
- Troubleshooting documentation for developers.

3.7 Git Workflow

The version control system for the project was Git and GitHub, and this was done using the collaborative branching model. Each task assigned was first done in a feature branch and later merged in the common staging branch after review. Below is an overview of the process that was followed during the internship.

Repository Initialization → Creation of a Feature Branch → Coding → Automated Testing → Manual Validation of APIs → Pull Request → Code Review → Feedback Review → Merging to Staging Branch

This process enabled traceability of changes in the code, collaboration, and quality of the codebase.

4. Results and Discussion

4.1 Overview

The internship activities allowed me to contribute to the implementation of the Life Programmable Interface (LPI) Platform, which is a modular backend application developed as part of the LifeAtlas ecosystem at WINNIIO AB. The tasks performed during the course of the internship aimed at enhancing the quality and stability of the platform through backend testing, documentation, integration assistance, debugging, and feature-specific development.

Through this internship, I gained experience in collaborative software engineering where the development process involved Github workflows, automated tests, code review, and continuous integration processes. My contributions were related to learning about the current architecture of the platform, validation of the backend system, automation of tests, helping with feature integration, developer documentation, and quality assurance activities for various modules of the platform.

A major accomplishment of this internship was completion of three verified pull requests that helped improve the functionality of the platform and software quality:

- PR #14 – SMILE Test Updates
- PR #28 – Increased Activity Signals Test Coverage and Time Range Querying Feature
- PR #34 – Recommendation Feedback Storage Using Supabase Storage

In addition to my involvement in coding the application, I was also involved in the final demo of the project and helped the team in testing, documentation, and integration efforts, which made the internship project a success.

4.2 Major Technical Contributions

- **Automation Testing Improvements** - One of the main contributions done during my internship was the improvement of the platform's automated testing system. Test cases already implemented were analyzed and run, and some additional test cases were created in order to better validate the work of the backend. Specifically, much effort was put into making sure that any newly added feature will not cause regression in already working parts of the system. The changes made with PR #14 helped improve the testing of the SMILE framework, and PR #28 extended the number of automated tests for Activity Signals module by creating new test cases and time-range queries support.
- **Feedback Persistence Implementation** - Yet another important contribution was the development of Recommendation Feedback Persistence using Supabase. With this feature, feedback given by users regarding the recommendations provided by the platform could be saved in the database of the platform. It was implemented in PR #34.
- **Backend Validation and API Testing** - Backend APIs were validated through automation and manual validation methods throughout the internship. Endpoints associated with Goals, Activity Signals, Recommendations, and other platform APIs were validated in order to check their request processing, response structure, and backend behavior. Manual validation of APIs through Swagger in FastAPI added to automated testing and assisted in finding bugs prior to code review.
- **Documentation** - Documentation was an integral part of the internship. Technical documentation was created and maintained to help in developer onboarding, repo configuration, and other processes. Documentation made it easier for us to collaborate and made things simpler for any developer joining the project.

4.3 Repository Improvements

Apart from the work on feature specific development, there were other efforts which aided the maintenance of the codebase of the project. They included:

- Working on test coverage.
- Working on testing documentation.
- Helping out in the documentation for repository setup.
- Checking for GitHub webhooks.
- Helping out in ZeroClaw documentation.
- Pull Request reviews and QA related tasks.

All these efforts helped in making the development process better.

4.4 Team Collaboration

The internship took place in a collaborative software engineering setting in which various developers worked on different modules of the system.

My responsibilities entailed working with my colleagues at different stages such as developing, testing, debugging, documenting, and reviewing the pull requests. The pair programming and collaborative debugging were done at any point that integration problems came up, especially when testing and integrating ZeroClaw. Getting feedback through peer reviews was beneficial not only in improving code quality but also in gaining insights in software engineering practices.

Being part of the final project presentation helped me present the finished product and see how my contributions fitted into the system.

4.5 Key Outcomes

Some of the tangible results that came from the internship were the following for the platform and personal development.

Technical Achievements

- Made contributions in the form of three confirmed pull requests.
- Increased the test coverage of the automated backend tests.
- Improved the validation of the Activity Signals module.
- Implemented the recommendation feedback persistence via Supabase.
- Worked on backend API validation.
- Worked on ZeroClaw integration validation.
- Contributions to developer documentation.
- Participation in quality assurance and debugging work.

Professional Achievements

- Got hands-on experience in collaborative software engineering.
- Developed experience in Git and GitHub development.
- Better knowledge of backend application architecture.
- Improved debugging and problem-solving skills.
- Learned modern testing approaches through pytest.
- Developed experience in technical documentation writing.

Area	Outcome
Backend Development	API validation and integration
Testing	Expanded automated coverage
Documentation	Setup guides and technical documentation
QA	Debugging and regression testing
Collaboration	GitHub workflow, PR reviews, final demo

4.6 Discussion

However, the internship helped understand that the successful creation of a software development platform that is production-oriented entails much more than just implementing features. The efficient operation of backend systems relies on proper architecture, testing, documentation, collaboration, and quality assurance in order to become truly maintainable.

Participation in the LPI Platform project gave me firsthand experience of how such engineering techniques were utilized during the software development process. Getting familiar with modular backend architecture, REST API creation, automation testing, versioning control, database integration, and code review taught me the importance of developing maintainable software and creating new functionality that would fit smoothly with the existing code.

While my personal contributions included mostly testing, documentation, backend validation, and feature development, they were parts of a larger effort towards enhancing the reliability and maintainability of the platform. Therefore, the internship was a great opportunity to apply my theoretical knowledge in practice and see how software engineering techniques were applied in the process of building AI-enabled backend systems.

5. Conclusion and Future Scope

5.1 Conclusion

The present internship has been an excellent learning experience in the areas of Artificial Intelligence, software engineering, and development through participation in developing the LPI Platform for WINNIIO AB. The 61-day long internship has involved being part of a multi-disciplinary development team working on production backend development, performing activities related to software testing, validation, debugging, quality assurance, documentation, and feature development.

The software development lifecycle from platform architecture and configuration of the local development environment to backend API validation, automated testing, repositories, documentation, and collaboration through GitHub was experienced during the internship. Working with technologies such as FastAPI, Supabase, Docker, Git, GitHub, and pytest has enabled me to gain a better understanding of modern backend applications development and software engineering.

One of the major tasks was making the platform more reliable and maintainable via testing and QA-related work. I contributed to the improvement of automated tests for the Activity Signals component, SMILE test cases, Recommendation Feedback Persistence with Supabase, ZeroClaw integration validation, GitHub webhook testing, and preparation of technical documentation to help other developers join the project and keep the repository clean. These contributions were properly implemented via pull requests and became a part of the effort of the team to improve the platform.

In addition to the technical work, I learned about the role of collaboration in large software development projects. Participating in architectural discussions, debugging, pull requests review, pair programming, and the demo of the project gave me valuable insights into how software engineering teams coordinate their work, ensure high quality, and incorporate separate contributions into a single codebase.

From a personal point of view, the internship helped me acquire substantial theoretical knowledge about the backend software development process and improve my analytical skills, debugging process, problem-solving skills, and the confidence in working in a collaborative environment. Thus, the internship gave me an excellent opportunity to connect theoretical knowledge acquired in the process of academic learning with practice and helped me prepare for the career in Artificial Intelligence and software engineering.

Despite the fact that the internship provided me with ample opportunity to learn the processes associated with the backend development and software engineering process, my personal contribution was limited to some particular modules and tasks assigned for development in the context of a collaborative process.

5.2 Future Scope

Based on the knowledge and experience gained in the course of the internship, the following enhancements can be proposed for the future development of the Life Programmable Interface (LPI) Platform:

- **Recommendation Intelligence Enhancement:** Upgrade the recommendation engine with more advanced approaches of AI and user feedback to create more customized and contextual recommendations.
- **Increase in Automated Testing:** Implement automated tests for all platform components and

add performance, integration, and security tests for increased software reliability.

- **Improvement of Developer Experience:** Develop technical documentation, reduce the complexity of the local developer environment and improve development processes.
- **Platform Scalability Strengthening:** Investigate cloud native approaches and optimization of backend infrastructure for better scalability and performance.
- **Platform Development Continuation:** Expand platform capabilities, provide integration opportunities and improve backend architecture.

6. References

- [1] FastAPI, “*FastAPI Documentation*”. [Online]. Available: <https://fastapi.tiangolo.com/>
- [2] Supabase Inc., “*Supabase Documentation*”. [Online]. Available: <https://supabase.com/docs>
- [3] Docker Inc., “*Docker Documentation*”. [Online]. Available: <https://docs.docker.com/>
- [4] Git SCM, “*Git Documentation*”. [Online]. Available: <https://git-scm.com/doc>
- [5] GitHub Inc., “*GitHub Docs*”. [Online]. Available: <https://docs.github.com/>
- [6] pytest Development Team, “*pytest Documentation*”. [Online]. Available: <https://docs.pytest.org/en/stable/>
- [7] LangChain Inc., “*LangGraph Documentation*”. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [8] Ollama, “*Ollama Documentation*”. [Online]. Available: <https://ollama.com/docs>
- [9] Python Software Foundation, “*Python 3 Documentation*”. [Online]. Available: <https://docs.python.org/3/>
- [10] Pydantic Team, “*Pydantic Documentation*”. [Online]. Available: <https://docs.pydantic.dev/latest/>
- [11] PostgreSQL Global Development Group, “*PostgreSQL Documentation*”. [Online]. Available: <https://www.postgresql.org/docs/>
- [12] OpenAPI Initiative, “*OpenAPI Specification*”. [Online]. Available: <https://spec.openapis.org/oas/latest.html>
- [13] Swagger, “*Swagger Documentation*”. [Online]. Available: <https://swagger.io/docs/>
- [14] Life Programmable Interface (LPI) Platform, “*Project README and Internal Technical Documentation*”, WINNIO AB, LifeAtlas Ecosystem, 2026. (*Internal project documentation used during the internship.*)